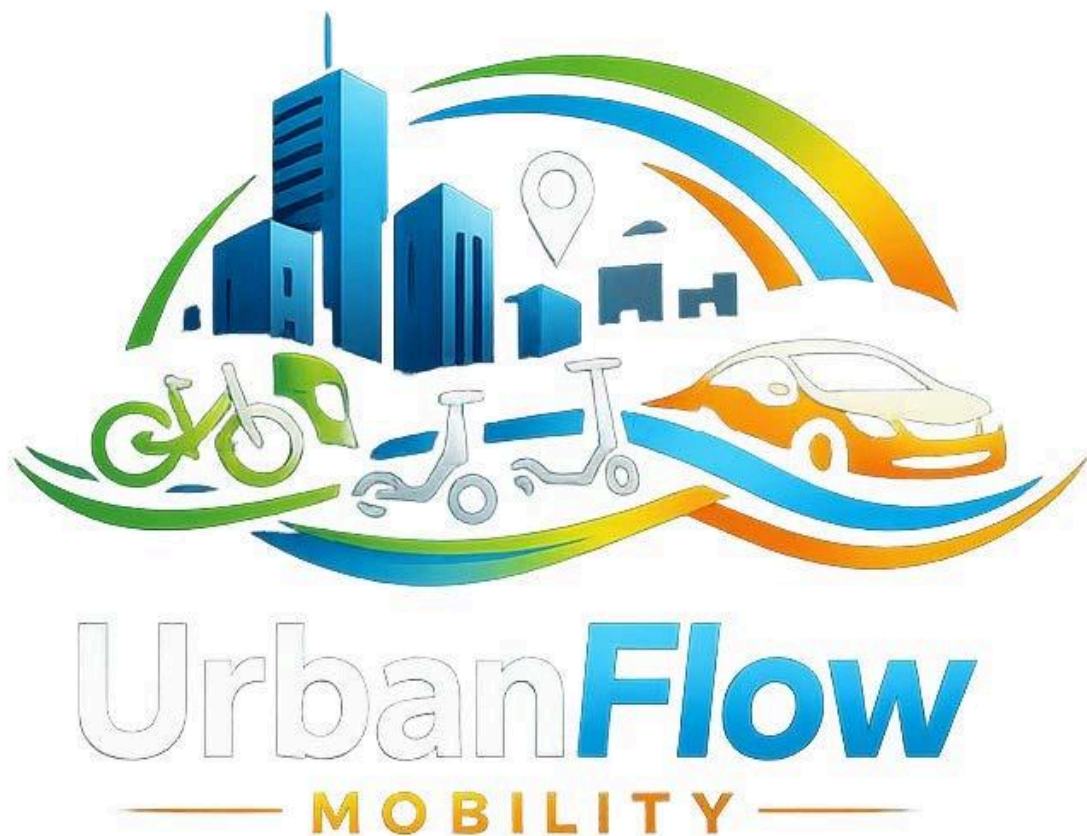


UrbanFlow Mobility



Eleisawy Mohamed

**Titre Professionnel – Concepteur Développeur de Solutions
Digitales (RNCP)**

Digital Campus - B3 Dev 2025-2026

UrbanFlow Mobility.....	1
1 - Présentation du projet.....	3
1.1 Présentation de la métropole de la donneuse d'ordre.....	4
1.2 Contexte.....	4
1.3 Analyse du besoin.....	5
1.4 Problématique.....	5
1.5 Objectifs du projet.....	6
1.6 Périmètre du projet.....	6
2 - Analyse et pilotage du projet.....	7
2.1 Le concept de MaaS (Mobility as a Service).....	7
2.1.1 Qu'est-ce qu'une plateforme MaaS ?.....	7
2.2 Benchmark concurrentiel.....	7
2.2.1 Rapport de veille concurrentielle.....	8
2.2.2 Tableau comparatif.....	10
2.2.3 Bilan du benchmark et positionnement stratégique.....	11
2.3 Analyse SWOT.....	11
2.4 Personas.....	12
2.5 Parcours utilisateur.....	13
2.6 Gestion de projet.....	15
2.6.1 Membres de l'équipe UrbanFlow Mobility.....	15
2.6.2 Méthodologie.....	17
2.7 Choix techniques.....	17
2.7.1 Méthodologie de sélection.....	18
2.7.2 Frontend.....	18
2.7.3 Backend.....	19
2.7.4 Choix du framework Microservice.....	20
2.7.5 Base de données.....	21
2.7.6 Redis.....	22
2.7.7 Docker.....	23
2.7.8 Synthèse des choix techniques.....	23
2.8 Identité visuelle.....	23
2.8.1 Objectifs graphiques.....	24
2.8.2 Moodboard.....	24
2.8.3 Ratios de contraste (WCAG).....	25
2.8.4 Typographie.....	25
2.9 Proposition financière.....	26
2.9.1 Estimation de la charge.....	26
2.9.2 Coûts logiciels et licences.....	26
2.9.3 Hébergement.....	27
2.9.4 Architecture d'hébergement.....	27
3 - Conception.....	27
3.1 Architecture globale.....	27
3.1.1 Frontend – Next.js (Vercel).....	28
3.1.2 Backend – NestJS.....	28

3.1.3 PostgreSQL + PostGIS.....	28
3.1.4 Redis (cache mémoire).....	29
3.1.5 APIs externes (GTFS, GTFS-RT, OSM, ADEME).....	29
3.2 Diagrammes.....	30
3.2.1 Diagramme de cas d'utilisation.....	30
3.2.2 Diagramme de Séquence.....	32
3.1.3 Diagramme de classes.....	33
3.3 Sécurité et conformité.....	34
3.4 Performances et éco-conception.....	35
4 - Bilan du projet.....	35
4.1 Difficultés rencontrées.....	36
4.2 Limites du projet.....	37
4.3 Perspectives d'évolution.....	37
5 - Conclusion.....	38
Webographie.....	39
Liens.....	39

1 - Présentation du projet

1.1 Présentation de la métropole de la donneuse d'ordre

Le projet est porté par une métropole française fictive de 500 000 habitants. Face aux enjeux de congestion routière, de pollution et de saturation des transports, la collectivité souhaite moderniser ses services publics pour accompagner la transition écologique de son territoire.

La direction et le pilotage de cette initiative sont confiés à **Madame Claire Hénette**, représentante du programme **UrbanFlow Mobility** et chargée de la conception de cette future plateforme citoyenne de mobilité. Son mandat couvre l'ensemble du cycle de vie du projet, structuré autour de la spécification des besoins, de la réalisation d'un MVP (*Minimum Viable Product*), puis du déploiement progressif de la solution sur l'ensemble du territoire métropolitain.

L'objectif central est de développer une plateforme numérique capable de centraliser toutes les options de transport locales. En permettant aux usagers de planifier facilement des itinéraires multimodaux, ce nouveau service public vise à encourager les mobilités douces et les transports en commun afin de réduire les émissions de gaz à effet de serre. Pour répondre pleinement aux attentes de la collectivité et des citoyens, l'application s'appuiera sur l'Open Data tout en garantissant des critères stricts de performance, d'accessibilité universelle, de sobriété numérique et de protection des données personnelles.

1.2 Contexte

Les grandes métropoles françaises font face à des défis majeurs : augmentation du trafic routier, saturation des réseaux de transport aux heures de pointe, pollution atmosphérique et impératif de réduction des émissions de CO₂. Pour faire face à ces défis, les politiques publiques poussent au développement de modes de transport plus écologiques (vélos, transports collectifs, autopartage).

Parallèlement, l'ouverture des données de transport (Open Data via les standards GTFS, GTFS-Realtime) offre aujourd'hui des opportunités inédites. Les citoyens, de leur côté, attendent des applications fiables, actualisées et capables de fournir des informations en temps réel pour comparer rapidement les différents modes de déplacement.

Cependant, les solutions de navigation existantes restent limitées. Elles se cantonnent souvent à la recherche d'itinéraires classiques basés sur le temps ou la distance, sans

proposer de vision globale intégrant la multimodalité, l'accessibilité universelle, le suivi personnalisé ou l'impact environnemental des trajets.

C'est précisément pour combler ce manque qu'a été imaginé **UrbanFlow Mobility** : une plateforme moderne capable d'accompagner les usagers dans leurs déplacements quotidiens tout en soutenant les objectifs de développement durable de la métropole en définissant un budget carbone personnel et d'exporter leurs données conformément aux exigences du RGPD.

1.3 Analyse du besoin

À l'issue de l'analyse du cahier des charges, plusieurs besoins ont été identifiés.

La collectivité souhaite promouvoir les mobilités douces en mettant davantage en valeur les transports en commun, la marche et le vélo. Elle souhaite enfin sensibiliser les usagers à l'impact environnemental de leurs déplacements. Les principaux besoins identifiés sont :

- *Un planificateur d'itinéraires multimodal intelligent qui combine tous les modes de transport disponibles*
- *Un système de réservation unifié pour vélos, trottinettes, parkings et places de covoiturage*
- *Une IA qui optimise les trajets en temps réel selon le trafic, la météo et les préférences utilisateur*

- *Un système de gamification avec récompenses pour les choix de mobilité durable*
- *Une plateforme de covoiturage dynamique pour les trajets quotidiens et ponctuels*
- *Un module de calcul et suivi de l'empreinte carbone des déplacements*
- *Des alertes et notifications personnalisées (perturbations, alternatives écologiques)*
- *Un tableau de bord citoyen pour visualiser ses habitudes de mobilité et leur impact*
- *Une fonctionnalité de signalement collaboratif des problèmes de voirie et transport*

1.4 Problématique

Un utilisateur peut consulter une première application pour rechercher un itinéraire, une seconde pour vérifier les perturbations du réseau et une troisième pour estimer l'impact environnemental de son déplacement. Cette fragmentation nuit à l'expérience utilisateur et limite l'adoption de comportements de mobilité plus responsables.

La problématique retenue est donc la suivante :

Comment concevoir une plateforme de mobilité intelligente capable de centraliser les informations de déplacement, de proposer des itinéraires multimodaux pertinents et d'encourager des choix de transport plus durables, tout en garantissant une expérience utilisateur simple, accessible, performante et conforme au RGPD ?

Cette problématique guidera l'ensemble des choix fonctionnels et techniques présentés dans les chapitres suivants.

1.5 Objectifs du projet

Objectif général

Développer une plateforme web de mobilité intelligente permettant aux habitants de préparer leurs déplacements de manière simple, rapide et responsable.

Objectifs spécifiques

- Centraliser les informations issues de plusieurs modes de transport
- Proposer une recherche d'itinéraires multimodaux
- Afficher les perturbations en temps réel
- Estimer les émissions de CO₂ de chaque trajet
- Garantir la conformité au RGPD et aux bonnes pratiques de cybersécurité
- Offrir une architecture évolutive facilitant l'ajout de nouvelles fonctionnalités

1.6 Périmètre du projet

Le périmètre fonctionnel comprend notamment :

- la recherche d'itinéraires multimodaux
- l'affichage cartographique des trajets
- la consultation des perturbations en temps réel
- le calcul de l'empreinte carbone
- la gestion des comptes utilisateurs
- le suivi des statistiques personnelles
- l'administration des données de mobilité

En revanche, certaines fonctionnalités ont volontairement été exclues du périmètre de cette première version afin de respecter le temps et la complexité du projet.

Parmi ces fonctionnalités figurent notamment :

- le paiement des titres de transport
- la réservation de vélos ou de véhicules partagés
- l'intégration d'un système de fidélité
- les notifications géolocalisées avancées

Ces éléments constituent des pistes d'évolution qui seront abordées dans la conclusion du rapport.

2 - Analyse et pilotage du projet

2.1 Le concept de MaaS (*Mobility as a Service*)

Avant de concevoir UrbanFlow Mobility, il était indispensable d'étudier les solutions déjà présentes sur le marché afin d'identifier les bonnes pratiques, les fonctionnalités les plus utilisées ainsi que les limites des applications existantes. Cette analyse permet de mieux positionner le projet et de définir les éléments différenciants qui apporteront une réelle valeur ajoutée aux futurs utilisateurs.

2.1.1 Qu'est-ce qu'une plateforme MaaS ?

Le concept de **Mobility as a Service (MaaS)** désigne une plateforme numérique capable de regrouper plusieurs services de mobilité au sein d'une seule application. Au lieu d'utiliser plusieurs applications différentes pour consulter les horaires, rechercher un itinéraire, réserver un transport ou acheter un billet, l'utilisateur dispose d'une interface unique lui permettant de gérer l'ensemble de ses déplacements. Les principales fonctionnalités d'une plateforme MaaS sont généralement :

- recherche d'itinéraires multimodaux
- consultation des horaires
- informations trafic en temps réel
- cartographie interactive
- réservation de services
- paiement intégré
- personnalisation des déplacements.

UrbanFlow Mobility s'inscrit dans cette approche en proposant une plateforme orientée vers la mobilité durable, tout en mettant l'accent sur l'accessibilité et l'impact environnemental des déplacements.

2.2 Benchmark concurrentiel

Afin d'identifier les meilleures pratiques du marché, plusieurs solutions de mobilité ont été étudiées.

Les critères retenus sont :

- informations temps réel
- mobilité multimodale
- calcul environnemental
- accessibilité

2.2.1 Rapport de veille concurrentielle

Google Maps:

- **Multimodalité:** itinéraires hybrides classiques (voiture + métro, bus + vélo). En revanche, elle n'intègre pas nativement tous les services : pour louer une trottinette ou réserver un VTC, elle redirige vers les applications partenaires.
- **Temps réel:** Excellent dans la gestion du trafic grâce à l'exploitation des flux ouverts des agences de transport (GTFS-RT). Il affiche les horaires en direct et notifie les incidents de manière très fiable.
- **Impact environnemental (CO₂) :** Propose des itinéraires écologiques (*Green Routing*) qui privilégient les trajets à moindre consommation de carburant, ce qui le positionne parmi les leaders sur les fonctionnalités de « mobilité verte ».
- **Paiement et billetterie (MaaS limité) :** L'application planifie le trajet mais ne permet pas l'achat direct de tickets de bus ou de métro.
- **Accessibilité :** Offre des options de base (compatibilité TalkBack/VoiceOver, guidage vocal) adaptées au grand public, mais ne garantit aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG).

Citymapper :

- **Multimodalité :** Il est pensé pour la combinaison. Elle intègre de manière exhaustive l'ensemble des options d'une ville : transports en commun (métro, bus, tram, train), mobilités douces (vélos et trottinettes partagés), marche, et VTC.
- **Temps réel :** Les calculs s'actualisent en direct grâce aux flux des agences locales, intégrant immédiatement les retards, incidents ou grèves.
- **Impact environnemental (CO₂) :** Citymapper ne propose pas nativement de calcul d'empreinte carbone pour comparer l'impact écologique des différents trajets.
- **Paiement et billetterie:** Permet de commander un VTC via des partenaires (Uber, Kapten), elle ne vend pas directement de titres de transport public.
- **Accessibilité :** Bien que son interface soit ergonomique, claire et très travaillée, l'entreprise ne revendique aucune conformité officielle aux normes d'accessibilité (comme les labels WCAG ou RGAA)

Moovit :

- **Multimodalité:** Combiner les transports en commun (bus, métro, tram, trains régionaux) avec la marche et le vélo (personnel ou partagé) et les trottinettes en libre-service. Elle intègre des services de VTC (Uber) pour les derniers kilomètres.
- **Temps réel:** Un système de guidage pas-à-pas en direct est très performant : il alerte précisément l'utilisateur lorsqu'il doit quitter son siège pour descendre ou se rendre à un arrêt. Ses données proviennent d'un modèle hybride associant les informations des opérateurs locaux et les signalements en direct de sa large communauté d'utilisateurs.
- **Impact environnemental (CO₂) :** Moovit intègre un indicateur d'empreinte carbone basé sur la distance et le mode de transport sélectionné. Cette fonctionnalité, bien que récente, lui donne une longueur d'avance écologique sur Citymapper.
- **Paiement et billetterie (MaaS limité) :** un modèle *freemium* « Moovit+ », qui offre quelques fonctionnalités avancées comme les cartes hors-ligne et alertes prioritaires.
- **Accessibilité :** Bien que son interface épurée soit pensée pour le grand public, l'éditeur ne communique sur aucune conformité officielle aux normes d'accessibilité numérique (RGAA ou WCAG).

SNCF Connect :

- **Multimodalité (Orientée interurbain) :** L'application est avant tout centrée sur le ferroviaire (TGV, TER) et les trajets longue distance (autocars, covoiturage). Bien qu'elle intègre les réseaux d'Île-de-France, elle éprouve des difficultés à proposer des combinaisons : train + bus local + vélo partagé.
- **Temps réel (Performant sur le rail) :** Très efficace pour le suivi du réseau ferré (retards, travaux, géolocalisation des trains et alertes in-app). En revanche, la gestion en direct des réseaux de bus urbains locaux ou des micro-mobilités n'est pas au cœur de son service, contrairement à des acteurs comme Moovit.
- **Impact environnemental (CO₂) :** Bien que le train soit écologique par nature, l'application ne propose pas d'outil poussé permettant de comparer l'empreinte carbone précise entre différents itinéraires multimodaux locaux.
- **Paiement et billetterie:** C'est le modèle à suivre. SNCF Connect est une véritable plateforme MaaS complète : elle permet de rechercher, réserver, payer (carte bancaire, Apple Pay) et stocker ses titres de transport de manière centralisée (billets de train, rechargement de cartes d'abonnement, tickets urbains en Île-de-France).
- **Accessibilité:** Malgré les engagements de la SNCF en matière de handicap, l'application ne met pas en avant de certification technique officielle (type RGAA ou WCAG), et son interface complexe a parfois été critiquée par les utilisateurs malvoyants.

Transit :

- **Multimodalité** : L'application se concentre exclusivement sur les transports en commun urbains (bus, métro, tram, train) combinés aux mobilités douces et partagées (vélos, trottinettes). Elle exclut volontairement la planification de trajets en voiture personnelle (hors liens vers Uber ou Waze), ce qui en fait un outil purement dédié à la mobilité urbaine alternative.
- **Temps réel** : L'application combine les infos officielles avec les signalements en direct des utilisateurs.
- **Impact environnemental (CO₂)** : Ne propose pas de calculateur d'empreinte carbone pour comparer les trajets.
- **Paielement et Billetterie (MaaS avancé)** : Permet l'achat de titres de transport et de pass pour les vélos en libre-service dans de nombreuses villes, se positionnant comme un acteur MaaS très abouti.
- **Accessibilité** : Bien que l'interface soit travaillée pour être simple et intègre des alertes par vibration, l'éditeur ne communique sur aucune certification d'accessibilité officielle (type WCAG).

2.2.2 Tableau comparatif

Solution	Recherche multimodale	Temps réel	Calcul CO ₂	Paielement	Accessibilité
Google Maps	Voiture, à pied, vélo, transports en commun, covoiturage mais pas de trottinettes ou réservation VTC	Excellent	Oui propose itinéraire économe en carburant	NON	Aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG) Mais offre la compatibilité TalkBack/ VoiceOver et guidage vocal
Citymapper	Ultra-multimodal (métro, bus, RER, train, taxis/VTC, vélos, trottinette...)	Oui réactualisation auto des itinéraires sur horaires en temps réel	Non	NON	Aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG)
Moovit	Multi (bus, RER, vélos, métro, covoiturage...)	Oui infos des opérateurs locaux et signalements de sa large	Oui	Moovit+ (cartes hors-ligne et alertes prioritaires)	Aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG)

		communauté			
SNCF Connect	train, TER, TGV, Eurostar ...) + transports locaux	Oui alertes trafic, géolocalisation en temps réel du train.	Non	Oui achat de billets (train, bus, tickets RATP/IDF, car, hôtel)	Aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG)
Transit	TC (bus, train, métro), avec bikeshare et trottinette.	Oui l'un des meilleurs temps réel (données officielles + crowdsourcing GO).	Non	Oui titres de transports et passes vélos	Aucune certification d'accessibilité officielle (comme le RGAA ou les normes WCAG)

Les analyses montrent que **Google Maps** et **Citymapper** dominent par leur multimodalité et leurs données en temps réel, tandis que des applis comme **Transit** ou **Moovit** excellent sur l'information instantanée et la couverture monde et **SNCF Connect** sur la billetterie ferroviaire et les mobilités liées à la SNCF.

2.2.3 Bilan du benchmark et positionnement stratégique

L'analyse comparative des solutions met en évidence plusieurs besoins encore insuffisamment couverts par les applications actuellement disponibles. Bien que chaque solution possède des points forts, aucune ne répond simultanément aux exigences de multimodalité, de transparence des données, d'engagement environnemental et d'accessibilité fixées par la donneuse d'ordre.

- **Multimodalité et temps réel ouverts** : Notre application combinera tous les modes de transport (transports en commun, vélo, marche, covoiturage) en exploitant les standards de données ouverts (GTFS, GTFS-RT et NeTEx). Elle offrira un guidage proactif avec des alertes personnalisées en cas de perturbations.
- **Engagement écologique natif** : Pour inciter les usagers aux mobilités douces, la plateforme intégrera au cœur de son algorithme un calcul poussé de l'empreinte CO2 pour chaque trajet, accompagné d'un tableau de bord environnemental personnalisé.
- **Inclusivité garantie (Accessibilité)** : Pour combler les lacunes des acteurs privés, UrbanFlow sera développée dès l'origine selon les normes strictes d'accessibilité numérique (standards WCAG / RGAA), garantissant un service utilisable par tous.
- **Éthique des données**: Contrairement aux modèles de Google ou Moovit basés sur la collecte de données. UrbanFlow proposera un service sans publicité sans forcer la création d'un compte utilisateur.

2.3 Analyse SWOT

Afin d'évaluer le positionnement du projet, une analyse SWOT a été réalisée.

Forces	Faiblesses
Utilisation des standards GTFS et GTFS-RT Calcul d'empreinte carbone Utilisation de technologies Open Source Application PWA Respect du RGPD Architecture modulaire	Dépendance aux données Open Data Temps de développement élevé Dépendance aux API tierces Projet ambitieux nécessitant une maintenance continue
Opportunités	Menaces
Transition écologique Open Data Encouragement des mobilités douces	Évolution ou arrêt des API partenaires Nouveaux concurrents

2.4 Personas

Persona 1 - Léna, 24 ans, étudiante en master

Profil : Léna est étudiante en master et réside dans le centre-ville. Elle ne possède pas de voiture et effectue la majorité de ses déplacements à l'aide des transports en commun et des mobilités douces. Sensible aux enjeux environnementaux, elle privilégie les solutions respectueuses de l'environnement.

Besoins

- Rechercher rapidement un itinéraire multimodal (vélo + tramway).
- Comparer le coût et la durée des différents trajets.
- Connaître l'impact environnemental de ses déplacements.
- Comparer plusieurs moyens de transport.
- Partager facilement un itinéraire avec ses amis.

Frustrations

- Indisponibilité des vélos en libre-service.

- Trottinettes hors service.
- Coûts supplémentaires non affichés.

«Je veux savoir en un coup d'œil combien ça me coûte et combien ça pollue.»

Persona 2 - Christiane, 67 ans, retraitée à mobilité réduite

Profil : Christiane vit seule et se déplace avec une canne. Elle utilise principalement les transports en commun mais rencontre régulièrement des difficultés liées à l'accessibilité des infrastructures.

Besoins

- Connaître l'accessibilité des stations.
- Être informée des ascenseurs hors service.
- Utiliser une interface simple avec de grands caractères.
- Bénéficier d'une lecture vocale des informations importantes.

Frustrations

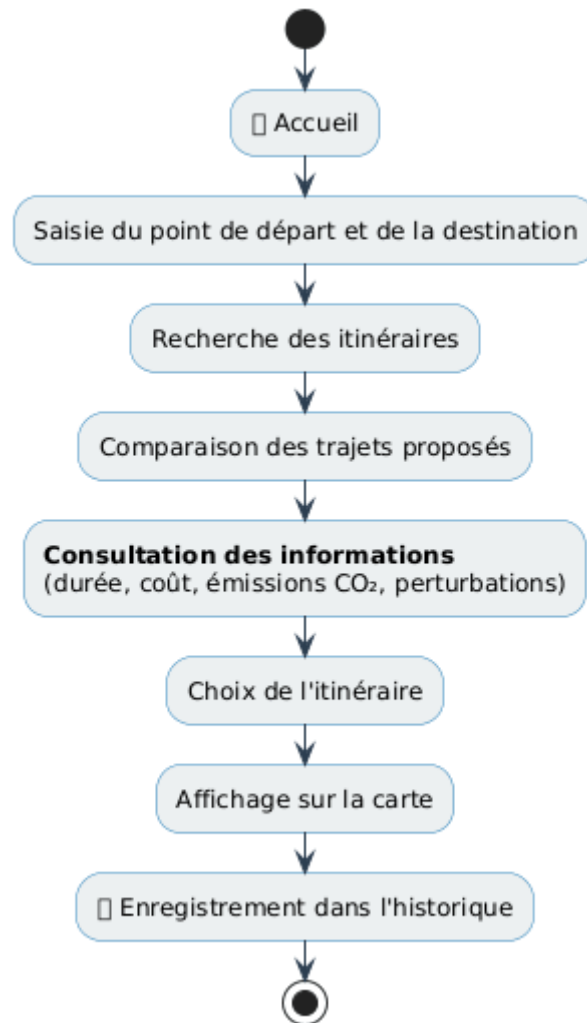
- Application complexes.
- Informations d'accessibilité incomplètes.
- Obstacles sur les trottoirs.

«J'ai besoin de savoir avant de partir si je vais pouvoir descendre du tram sans aide.»

Ces différents profils mettent en évidence des attentes variées : simplicité d'utilisation, rapidité d'accès à l'information, personnalisation des trajets, accessibilité et prise en compte de l'impact environnemental. Ils ont constitué un fil conducteur lors de la conception des fonctionnalités d'UrbanFlow Mobility afin de proposer une solution adaptée à un large public.

2.5 Parcours utilisateur

Après avoir identifié les différents profils d'utilisateurs, il est nécessaire d'analyser la manière dont ils interagiront avec la plateforme. Le parcours utilisateur permet de visualiser les principales étapes suivies par un usager afin d'atteindre son objectif.



Description des étapes :

Étape	Description
Accueil	L'utilisateur accède à la plateforme et saisit son trajet
Recherche	L'application interroge les différentes sources de données afin de calculer les itinéraires disponibles
Comparaison	Les itinéraires sont comparés selon la durée, le coût, le nombre de correspondances et l'empreinte carbone
Consultation	L'utilisateur consulte les détails du trajet ainsi que les éventuelles perturbations en temps réel.

Sélection	L'itinéraire retenu est affiché sur la carte interactive
Historique	Les utilisateurs authentifiés peuvent enregistrer leur trajet et consulter leur historique

2.6 Gestion de projet

2.6.1 Membres de l'équipe UrbanFlow Mobility

Chef de projet : Mohamed Eleisawy

- **Formation** : Bachelor Concepteur Développeur de Solutions Digitales (Titre RNCP 6), Digital Campus Paris
- **Profil** : Chef de projet à double casquette (gestion produit + développement full-stack). Pilote la vision globale du projet et veille à la cohérence technique et fonctionnelle.
- **Compétences clés** : Gestion de projet Agile, analyse des besoins, architecture logicielle, développement full-stack (JavaScript/TypeScript, Python), sensibilisation à l'UX.
- **Responsabilités** :
 - Cadrage des besoins avec la donneuse d'ordre.
 - Rédaction des user stories
 - Pilotage du planning, du budget et suivi des risques.
 - Arbitre technique et fonctionnel, garant du respect des exigences (PWA, RGPD, WCAG, éco-conception, etc.).
 - Coordination de l'équipe (UI/UX, frontend, backend) et alignement des livrables.
- **Outils maîtrisés** : GitHub, Notion, Figma.

UI/UX Designer : Karim Benali

- **Formation** : Bachelor Design
- **Profil** : Designer UX/UI orienté accessibilité. Expérience en agence. passionné par l'éco-conception et la simplicité d'usage.
- **Compétences clés** : Recherche utilisateur (interviews, tests), architecture de l'information, prototypage (wireframes, maquettes), design system, design mobile-first, audit d'accessibilité (WCAG/RGAA).
- **Responsabilités** :
 - Réalisation des personas et des tests utilisateurs.
 - Conception des wireframes et maquettes.

- Définition d'un design system complet (couleurs, typographie, composants, états interactifs).
- Audit d'accessibilité continu pour conformité WCAG.
- **Outils maîtrisés :** Figma, Storybook, plug-ins d'accessibilité (Stark), Lookback pour tests utilisateurs.

Développeuse Frontend : Sofia Garcia

- **Formation :** Diplômée EPITECH, spécialisation développement web.
- **Profil :** Développeuse frontend experte en React/Next.js. Sensible à la performance, à l'optimisation énergétique des interfaces et à l'expérience utilisateur fluide.
- **Compétences clés :** Intégration des maquettes en composants réactifs, mise en place d'une Progressive Web App, intégration d'APIs REST..
- **Responsabilités :**
 - Implémentation des écrans et composants Next.js à partir des maquettes Figma.
 - Mise en place de la PWA
 - Consommation des APIs backend (itinéraires, calcul CO₂, alertes) et données multimodales
 - Optimisation des performances front-end (minimisation des bundles, lazy loading).
 - Tests unitaires
 - Revue de code frontend et veille technologique (React, éco-conception).
- **Outils maîtrisés :** Next.js, TypeScript, Tailwind CSS, Lighthouse.

Développeur Backend / DevOps : Théo Dubois

- **Formation :** Formation bootcamp Node.js / Python, autodidacte en cloud et DevOps.
- **Profil :** Développeur backend. Expérience en NestJS et Python pour le traitement de données et l'automatisation. Sensible à la robustesse et à la maintenabilité.
- **Compétences clés :** Architecture backend, base de données SQL, API REST, intégration continue.
- **Responsabilités :**
 - Conception de l'architecture logicielle.
 - Création et migration du schéma de la base de données.
 - Développement de l'API REST (NestJS) pour la gestion des utilisateurs, des itinéraires et des données trafic.
 - Intégration des sources de données externes : GTFS, GTFS-RT (transports publics), API trottinettes/vélos en libre-service.
 - Conformité RGPD.
 - Déploiement

- **Outils maîtrisés :** NestJS, FastAPI, PostgreSQL, Redis, Prisma (ORM), Docker, GitHub Actions.

2.6.2 Méthodologie

Le projet a été conduit selon une méthode **Agile**, reposant sur un développement itératif et incrémental.

Cette approche nous permet de :

- découper le projet en fonctionnalités.
- prioriser les besoins du commanditaire.
- intégrer progressivement les nouvelles fonctionnalités.
- corriger rapidement les anomalies identifiées.

Organisation des sprints : .

Chaque sprint comprenait les étapes suivantes :

1. Planification des objectifs
2. Développement des fonctionnalités
3. Tests
4. Démonstration
5. Rétrospective

Outils utilisés :

Outil	Utilisation
Figma	Maquettes
GitHub	dépôt et collaboration et suivi des tâches
Draw.io	Diagrammes UML
Docker	Environnement de développement

2.7 Choix techniques

Le choix des technologies constitue une étape déterminante dans la réussite d'un projet logiciel. Cette section présente les principales technologies utilisées dans UrbanFlow Mobility, les alternatives étudiées ainsi que les raisons ayant conduit aux choix retenus.

2.7.1 Méthodologie de sélection

Les solutions retenues devaient répondre aux contraintes suivantes :

- être open source ou largement adoptées par la communauté
- offrir de bonnes performances
- faciliter la maintenance du projet
- disposer d'une documentation complète
- permettre une évolution future du projet.

2.7.2 Frontend

Le frontend constitue le point d'entrée principal de l'utilisateur. Il devait répondre à plusieurs contraintes :

- Interface moderne
- Expérience utilisateur
- Rapidité de chargement
- Référencement optimisé
- Compatibilité PWA

Plusieurs frameworks ont été étudiés :

Critère	Next (choisi) ✓	React (Vite)	Angular	Vue.js
Performance	utilise le SSR (rendu côté serveur) quasi immédiat . Très bien	SPA par défaut. Prends plus de temps au démarrage. Bien	SPA mais le framework est plus lourd. Moyenne	SPA par défaut. Prends plus de temps au démarrage. Bien
SEO	le serveur envoie une page HTML complète Très bien	Page envoyée par le serveur est vide : <code><div id="root"></div></code> Faible	Même que React. Possible d'améliorer cela avec <i>Angular Universal</i> . Moyenne	Page envoyée par le serveur est vide : <code><div id="root"></div></code> Faible
Routing intégré	Le système est inclus. Très bien	Il faut installer et configurer le module <i>React Router</i> . Faible (Via librairie externe)	Le système est inclus Très bien	Il faut installer et configurer le module <i>Vue Router</i> . Faible (Via librairie externe)

SSR / SSG	Le framework a été littéralement inventé pour ça. Très bien	besoin de next. Non	Il faut installer et configurer <i>Angular</i> <i>Universal</i> Moyenne	Il faut installer et configurer <i>Nuxt.js</i> Moyenne
PWA	Oui	Oui	Oui	Oui
Écosystème	Bon écosystème utilise React	Technologie frontend la plus utilisée au monde	Très présent, dans les vieux systèmes bancaires ou les énormes systèmes d'entreprise	Très populaire et apprécié, mais possède une part de marché professionnelle inférieure à React

Pourquoi Next.js ? :

Next.js a été retenu car il offre :

- un Très bon niveau de performance
- un système de routage intégré
- le rendu côté serveur (SSR) et la génération statique (SSG)
- une excellente compatibilité avec React
- une intégration facilitée des Progressive Web Apps

Ces caractéristiques répondent aux exigences du projet, notamment en matière de rapidité, d'expérience utilisateur et de référencement.

2.7.3 Backend

Le backend assure l'ensemble de la logique métier de la plateforme.

Il est responsable :

- de l'authentification
- de la gestion des utilisateurs
- du calcul des itinéraires
- de la communication avec les services externes
- de l'exposition des API

Critère	NestJS (choisi) ✓	Express	Django
Architecture	Structurée (Orientée Objet)	Peut coder toute l'application dans un seul fichier Minimaliste / Libre	Structure (MVC) conçu pour créer des sites lourds
TypeScript	Il a été créé avec et pour TypeScript. Le typage est omniprésent. Natif	Partiel (Configuration requise)	Non
Injection de dépendances	Native et intégrée	Non	Ne permet pas de simuler facilement la connexion à des services externes
Modularité	Fonctionne par modules isolés. Très bien	Faible (À concevoir manuellement)	Utilise le système "d'Apps" Bonne

Décision :

Next.js a été retenu car il permet de bénéficier d'une architecture moderne, de performances optimisées et d'une expérience utilisateur fluide.

2.7.4 Choix du framework Microservice

Le calcul de l'empreinte carbone environnementale nécessite des traitements mathématiques. Au lieu d'utiliser l'API principale (NestJS), cette fonctionnalité a été isolée sous la forme d'un **microservice développé en Python**

Cette architecture décentralisée présente plusieurs atouts stratégiques :

- Indépendance du moteur de calcul (isolé du reste de l'application).
- Possibilité de faire évoluer les algorithmes environnementaux sans risquer de créer des bugs sur le backend principal.
- Réutilisation future du service (la métropole pourra offrir ce microservice CO₂ à d'autres applications partenaires).

Pour développer cette brique indépendante, trois frameworks Python ont été comparés :

Critère	FastAPI (choisi) ✓	Flask	Django
Performance	fait plusieurs choses à la fois. Rapide &	fonctionne par file. Lors d'un pic d'affluence,	il traite les demandes à la suite (synchrone) + lourd

	asynchrone. Très Bien	l'application ralentit. Moyen	
Typage	utilise <i>Pydantic</i> on définit une règle si elle n'est pas valide FastAPI bloque la demande tout de suite et renvoie un message d'erreur clair. Pas de plantage. Strict	Le système est très permissif. Non	Le système est très permissif. Non
API REST	Tout son moteur est optimisé pour recevoir des données brutes, faire un calcul et renvoyer le résultat à la machine qui a posé la question. Spécialisé	On peut créer une API avec, mais il faut chercher, installer et assembler plusieurs modules supplémentaires pour obtenir un bon résultat. Adaptable	il faut lui greffer un énorme outil supplémentaire (<i>Django Rest Framework</i>) Via librairie externe

Décision :

FastAPI a été retenu car il offre la combinaison parfaite entre des performances d'exécution, une sécurité absolue des données grâce au typage strict.

2.7.5 Base de données

La plateforme UrbanFlow Mobility manipule quotidiennement des données complexes et fortement interconnectées : profils d'utilisateurs, historiques d'itinéraires, emplacements des stations, coordonnées GPS en temps réel et cartographie urbaine. Pour garantir la cohérence de ces informations une comparaison a été menée en se concentrant sur une exigence critique pour une application de mobilité : le traitement des données spatiales.

Critère	PostgreSQL	MySQL	MongoDB
SQL	Relationnel (SQL)	Relationnel (SQL)	Orienté Document (NoSQL)
Données géographiques	C'est l'outil utilisé par tous les grands acteurs de la cartographie	Suffisant pour une application classique qui a juste besoin	Excellent pour le temps réel très simple mais comme il n'est pas

	(OpenStreetMap, IGN, et la plupart des calculateurs d'itinéraires). Propose des centaines de fonctions géographiques. Standard	d'afficher la boutique la plus proche sur une carte, mais totalement insuffisant pour gérer un réseau de transports en commun métropolitain. Minimum syndical	relationnel il est incapable de croiser des données complexes ex: <i>"Croise les horaires des bus avec les limites géographiques de cette zone de travaux"</i> . Rapide mais limité
PostGIS	Oui	Non , n'est pas compatible avec PostGIS.	Non , n'est pas compatible avec PostGIS.
Intégrité des données	Il vérifie chaque donnée qui entre Rigoureux	Permissif afin de ne pas bloquer les applications. Si un système tente d'enregistrer un texte légèrement trop long, MySQL a tendance à le couper silencieusement pour le faire rentrer dans la case.	Par défaut, accepte presque tout. Si on envoie le mauvais format de données, MongoDB l'enregistre quand même, ce qui crée des bugs plus tard. Souple

Décision :

Le choix s'est porté sur **PostgreSQL**. En tant que base de données relationnelle, elle garantit une intégrité absolue pour relier les données de la plateforme (utilisateurs, stations, trajets). Son atout décisif face à MySQL et MongoDB est son extension spatiale **PostGIS** qui permet notamment d'effectuer des calculs géographiques avancés, indispensables pour la gestion des itinéraires.

2.7.6 Redis

Certaines données sont consultées très fréquemment :

- stations
- horaires
- perturbations

Redis est utilisé afin de :

- éduire les temps de réponse

- limiter les appels aux API externes

2.7.7 Docker

Le projet est entièrement conteneurisé. Docker permet de :

- reproduire facilement l'environnement
- simplifier le déploiement
- éviter les problèmes liés aux différences entre systèmes ;
- faciliter le travail collaboratif

2.7.8 Synthèse des choix techniques

Technologie retenue	Justification principale
Next.js	Performances, SEO, PWA et expérience utilisateur
NestJS	Architecture modulaire, TypeScript, maintenabilité
Redis	Réduction des temps de réponse
FastAPI	Performance et séparation des responsabilités
PostgreSQL + PostGIS	Gestion avancée des données géographiques
Docker	Portabilité et reproductibilité des environnements
GitHub Actions	Automatisation des tests et du déploiement

2.8 Identité visuelle

L'identité visuelle constitue un élément essentiel de l'expérience utilisateur. Elle permet d'assurer la cohérence graphique de l'application, de renforcer la reconnaissance de la marque UrbanFlow Mobility et de faciliter la lecture des informations. Les choix graphiques réalisés répondent aux objectifs de simplicité, d'accessibilité et de mise en valeur des données liées à la mobilité durable.

2.8.1 Objectifs graphiques

L'identité visuelle a été conçue autour de quatre objectifs principaux :

- inspirer confiance grâce à une interface moderne et épurée
- mettre en avant les mobilités durables à travers une palette de couleurs évoquant l'environnement
- faciliter la lecture des informations grâce à une hiérarchie visuelle claire
- garantir une accessibilité conforme aux recommandations WCAG, notamment en matière de contraste des couleurs.

2.8.2 Moodboard

Couleurs principales :

#1e3a5F En-têtes, navigation, CTA principaux, bordures clés	#2D7D46 Indicateurs éco-score, boutons secondaires, badges CO ₂
#F5F5F5 Fond de page, cartes, composants	#1A1A2E Mode sombre (optionnel), titres forts

Le bleu (#1E3A5F) : Le bleu constitue la couleur dominante de l'application. Il est associé à la confiance, à la stabilité et à la technologie, il est utilisé pour les éléments de navigation et les appels à l'action principaux.

Le vert (#2D7D46) : Le vert symbolise les mobilités durables. Il est principalement utilisé pour afficher les indicateurs liés à l'empreinte carbone, l'éco-score et les validations.

Le gris clair (#F5F5F5) : Cette couleur est utilisée comme fond principal afin de limiter la fatigue visuelle.

Le bleu nuit (#1A1A2E) : Pour les titres et le mode sombre, il apporte du contraste tout en conservant une cohérence avec la couleur principale.

Couleurs Secondaires :

#38A169 Confirmations, itinéraires optimaux	#718096 Textes secondaires, labels, métadonnées
#FF8C00 Warnings, notes, retards modérés	#E53E3E Erreurs et champs invalides

2.8.3 Ratios de contraste (WCAG)

Combinaison	Ratio	Conformité
#1E3A5F sur #F5F5F5	10.55	AAA
#2D7D46 sur #FFFFFF	5.08	AA
#1A1A2E sur #F5F5F5	15.65	AAA

2.8.4 Typographie

La typographie retenue est **Geist**, une police développée par Vercel et optimisée pour les applications construites avec **Next.js**. D'une part, car elle offre une bonne lisibilité ce qui améliore l'expérience utilisateur. D'autre part, la police est intégrée localement grâce au module **next/font/local**, ce qui présente plusieurs avantages :

- aucune requête vers un service externe comme Google Fonts
- amélioration des performances de chargement de l'application
- suppression d'une dépendance à un service tiers
- meilleure conformité avec le RGPD grâce à l'absence de transmission de l'adresse IP des utilisateurs vers des serveurs externes.

Ce choix s'inscrit dans les objectifs du projet, qui vise à développer une application performante, respectueuse de la vie privée et conforme aux bonnes pratiques du dev web moderne. L'identité visuelle a été conçue en tenant compte des recommandations du **RGAA** et des **WCAG** afin d'améliorer la lisibilité des contenus et de faciliter l'utilisation de l'application par les personnes présentant des déficiences visuelles.

2.9 Proposition financière

2.9.1 Estimation de la charge

Poste	Jours	TJM	Total
Chef de projet	12	600 €	7 200 €
UX/UI Designer	10	550 €	5 000 €
Développeur Frontend	20	550 €	11 000 €
Développeur Backend	22	600 €	13 200 €

2.9.2 Coûts logiciels et licences

Élément	Coût
GitHub	Gratuit
PostgreSQL	Gratuit
Redis	Gratuit
Docker	Gratuit
Next.js	Gratuit
NestJS	Gratuit
FastAPI	Gratuit

2.9.3 Hébergement

Élément	Fournisseur	Coût
Frontend	Vercel	0 €
VPS	OVHcloud	15 €/mois
PostgreSQL	Inclus sur le VPS	0 €
Redis	Inclus sur le VPS	0 €
Docker	Inclus	0 €
Nom de domaine	OVHcloud	~12 €/an
Certificat SSL	Let's Encrypt	0 €

2.9.4 Architecture d'hébergement

1. L'affichage (Utilisateurs → Vercel)

L'utilisateur ouvre l'application sur son téléphone. Son téléphone se connecte à **Vercel**. Vercel est surpuissant pour afficher instantanément la partie visuelle (le bouton de recherche, les couleurs, l'interface vide) grâce à [Next.js](https://nextjs.org/).

2. La demande (Vercel → OVHcloud)

L'utilisateur tape sa recherche et clique sur "Chercher". L'interface (Vercel) envoie alors un message sécurisé (HTTPS) à votre serveur privé chez **OVHcloud** pour calculer le trajet.

3. À l'intérieur d'OVHcloud

- **NestJS** : Il reçoit la demande. C'est lui qui va diriger les opérations.
- **La carte (PostgreSQL/PostGIS)** : NestJS interroge la base de données sur les stations de tram et les routes entre la Gare et le Centre-ville
- **Le temps réel (Redis)** : NestJS interroge le cache mémoire s'il y a un retard ou une panne sur ces lignes en ce moment.
- **L'expert écolo (FastAPI)** : NestJS envoie les kilomètres trouvés à votre microservice Python puis calcule l'empreinte CO₂ exacte de ce trajet.

3 - Conception

3.1 Architecture globale

3.1.1 Frontend – Next.js (Vercel)

Le frontend Next.js fournit l'interface utilisateur : il rend dynamiquement les pages React avec SSR/SSG selon le contexte, améliore le SEO et prend en charge la PWA. Il gère également l'authentification (JWT). Toutes les opérations serveur (recherche d'itinéraire, requêtes utilisateur, etc.) se font via des appels HTTPS REST vers NestJS. Next.js utilise le **next.config.js** pour définir les variables d'environnement. L'hébergement sur Vercel est gratuit et permet un déploiement continu à chaque push GitHub. On configure CORS côté NestJS pour limiter les origines autorisées au domaine Vercel. Pour la production, on peut passer au plan **Pro** de Vercel (~20 \$/mois) pour garantir des temps de réponse très courts. La documentation Next.js dit que ce framework permet de construire des applications web « full-stack » robustes et optimisées pour la performance.

3.1.2 Backend – NestJS

Le service **NestJS** (hébergé en conteneur Docker sur le VPS OVH) sert de passerelle API. Ses responsabilités sont l'authentification (login, gestion des JWT, rôles utilisateur), la validation des requêtes, la coordination des appels vers la base de données et le formatage des réponses JSON. Par exemple, l'endpoint **GET /api/itineraires** extrait de la requête le point de départ et d'arrivée, vérifie d'abord dans Redis s'il existe un itinéraire en cache, puis en cas de cache miss interroge PostGIS (calcul d'itinéraire géospatial via requêtes SQL sur les tables de réseau) ainsi que les flux GTFS-RT pour ajouter les perturbations en temps réel. Le résultat est mis en cache dans Redis pour les requêtes ultérieures. D'autres endpoints gèrent :

- **Calcul CO₂** (POST /api/carbone) délègue à FastAPI le calcul des émissions du trajet spécifié.
- **Gestion du profil/utilisateur** (GET /POST /api/users) création, mise à jour d'un profil.
- **Alertes/Notifications** (GET /api/alerts, POST /api/alerts/ack) gestion des messages d'alerte liés au compte ou au trafic.
- **Administration** (POST /api/admin/*) endpoints protégés pour charger les fichiers GTFS statiques, gérer les données (CRUD) du réseau de transport.

Chaque endpoint est défini via les contrôleurs NestJS, qui utilisent Express. Les en-têtes CORS sont configurés pour n'accepter que les requêtes du domaine Vercel du frontend, évitant ainsi les appels malveillants. Côté monitoring, on collecte les logs d'application.

3.1.3 PostgreSQL + PostGIS

La base **PostgreSQL** (avec extension **PostGIS**) stocke toutes les données persistantes :

- **Données métiers spatiales : tables des arrêts** de transport, des lignes et itinéraires pour le calcul d'itinéraire. PostGIS permet de stocker des géométries (points, polygones) et d'effectuer des requêtes spatiales (calcul d'itinéraires, distance minimale) très efficacement. Par exemple, on peut utiliser la fonction **ST_Distance** pour déterminer le chemin le plus court entre deux points sur un graphe routier modélisé.
- **Données utilisateurs : table users** (profil, email, mot de passe haché), user_preferences (modes favoris, accessibilité), carbon_budgets (budget annuel carbone), history (trajets enregistrés).

Les connexions à PostgreSQL s'effectuent sur le port standard 5432 (accessible depuis le VPS OVH uniquement). On utilisera le module pg ou typeorm/Prisma dans NestJS, et asyncpg ou SQLAlchemy dans FastAPI. Pour PostGIS, la documentation indique qu'il permet d'accélérer les requêtes de géométrie.

3.1.4 Redis (cache mémoire)

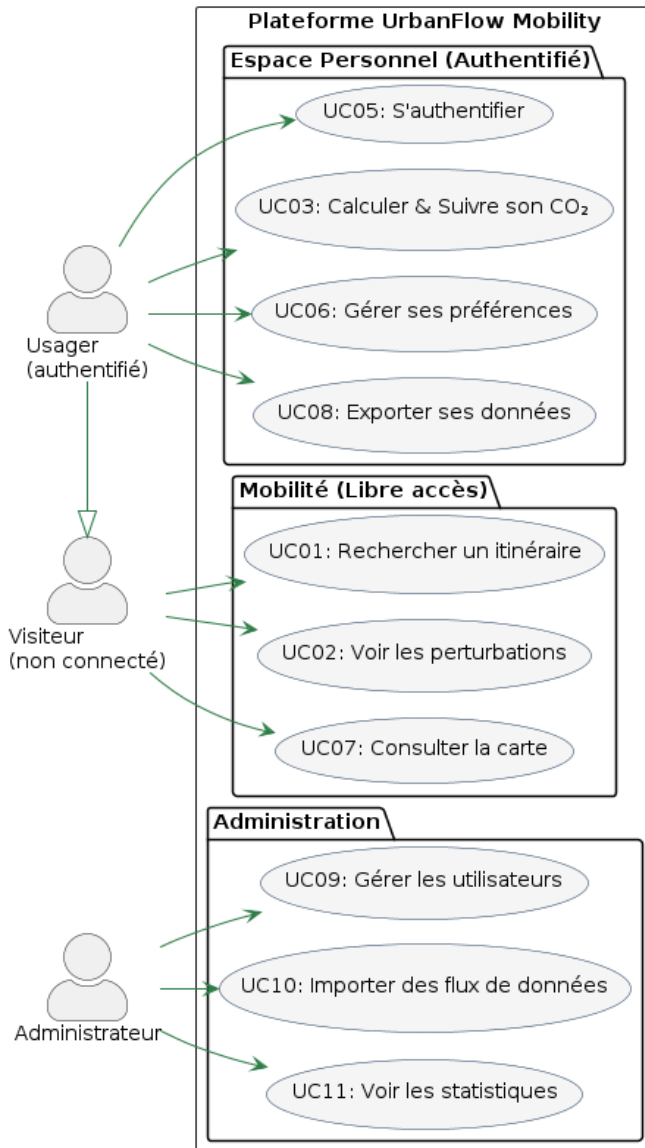
utilisé comme cache en mémoire pour améliorer les performances des lectures intensives (calculs d'itinéraires récurrents, tokens JWT, sessions). Dans notre architecture, Redis stocke les résultats d'itinéraires récents (GET /api/itinerares). La connexion se fait sur le port 6379 depuis NestJS (configuré via REDIS_URL). La documentation Redis recommande l'usage de Redis pour optimiser les performances des applications web (réduction du temps de réponse) grâce à son accès très rapide en mémoire.

3.1.5 APIs externes (GTFS, GTFS-RT, OSM, ADEME)

- **GTFS & GTFS-RT** : Fournit les fichiers statiques (.txt) contenant l'intégralité des horaires et arrêts des transports en commun d'une région. Ces fichiers sont publiés par la métropole ou la société de transport sur une API ou une URL fixe. Dans NestJS, un service d'importateur télécharge périodiquement ces fichiers (.txt) compressés et les charge dans les tables Postgres pour alimenter le calcul d'itinéraires. GTFS-RT fournit en temps réel les retards, annulations et positions des véhicules. À chaque calcul d'itinéraire, l'API NestJS interroge le flux GTFS-RT pour ajuster les horaires et avertir l'utilisateur.
- **OpenStreetMap (OSM)** : OSM fournit des données cartographiques et géographiques libres. Nous utilisons OSM pour le rendu cartographique.
- **Base Carbone ADEME** : L'ADEME met à disposition une **API Base Carbone** pour récupérer les facteurs d'émission CO2. Cette API REST est ouverte, documentée et gratuite.

3.2 Diagrammes

3.2.1 Diagramme de cas d'utilisation



Le système interagit avec trois profils d'utilisateurs :

- **Le Visiteur (non connecté) :** C'est l'utilisateur de base. Il accède librement aux services publics d'information de la plateforme sans avoir besoin de créer un compte.
- **L'Usager (authentifié) :** C'est un visiteur qui a créé un compte. Il conserve l'accès aux fonctions de base (recherche d'itinéraires, carte), mais débloque l'accès exclusif à son espace personnel pour le suivi de son empreinte carbone et la gestion de ses préférences.
- **L'Administrateur :** C'est un membre de l'équipe technique ou métier de la métropole, chargé de superviser le bon fonctionnement de la plateforme.

Les fonctionnalités ont été regroupées en trois grands blocs logiques, correspondant aux différentes zones de l'application.

Mobilité (Libre accès)

Ce module représente le cœur public de l'application, accessible à tous dès la page d'accueil.

- **Rechercher un itinéraire** : Permet au visiteur de saisir un point de départ et une destination pour obtenir les meilleurs trajets combinant différents modes de transport (bus, vélo, marche).
- **Voir les perturbations** : Affiche la liste des alertes en cours sur le réseau (retards, pannes, travaux) pour informer les voyageurs.
- **Consulter la carte** : Offre une vue interactive permettant de localiser visuellement les stations, le tracé des lignes et les véhicules disponibles en temps réel.

Espace Personnel (Authentifié)

Ce module est le coffre-fort de l'utilisateur. Il regroupe toutes les fonctionnalités nécessitant d'identifier la personne.

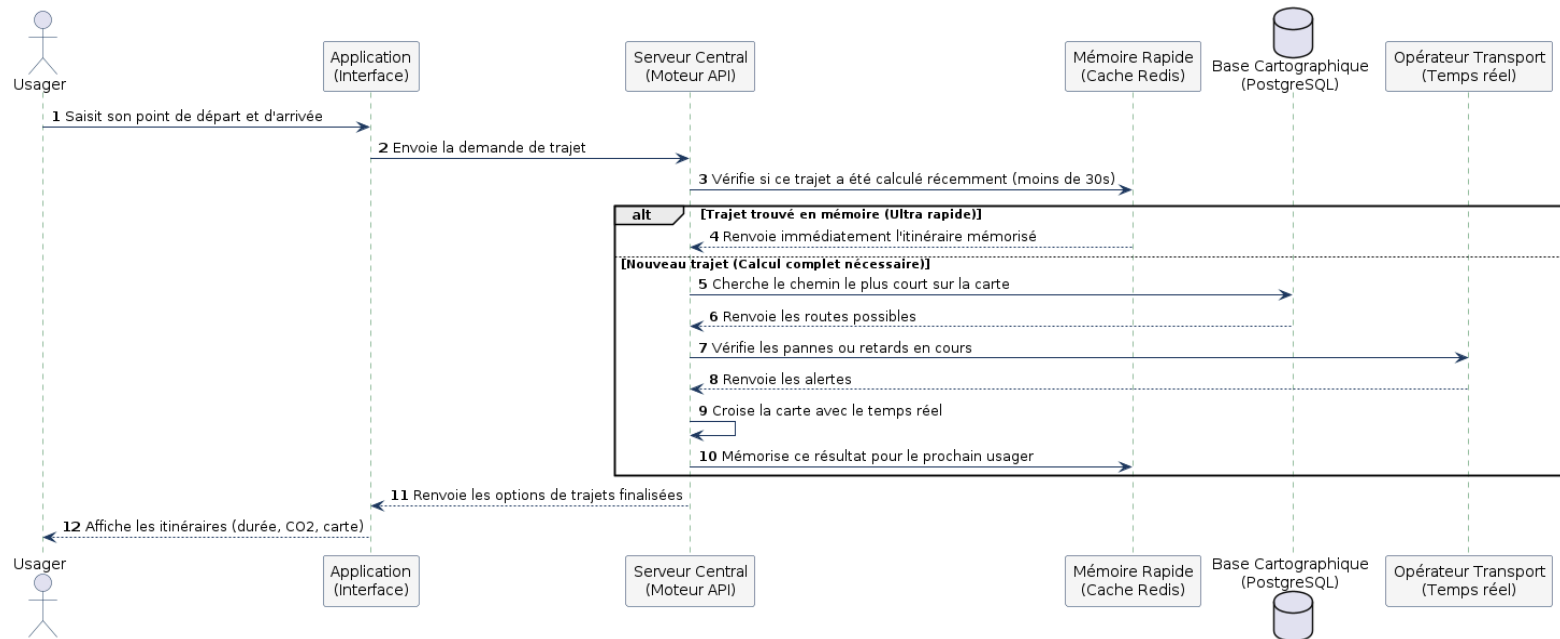
- **S'authentifier** : Permet à l'utilisateur de se connecter de manière sécurisée à son espace.
- **Calculer & Suivre son CO₂** : Évalue automatiquement l'impact écologique des trajets réalisés et permet à l'utilisateur de visualiser ses économies d'émissions via un tableau de bord.
- **Gérer ses préférences** : Permet de mémoriser des adresses favorites (Domicile, Travail) ou de définir un mode de transport privilégié pour accélérer les recherches.
- **Exporter ses données** : Garantit la conformité au RGPD en offrant à l'utilisateur la possibilité de télécharger à tout moment un fichier contenant l'intégralité de ses informations personnelles.

Administration

Ce module est totalement invisible pour le grand public. Il sert d'interface de pilotage (Back-office) pour la métropole.

- **Gérer les utilisateurs** : Interface de modération permettant d'assister les usagers (réinitialisation de mots de passe, suppression de comptes).
- **Importer des flux de données** : Fonction permettant de mettre à jour manuellement la base de données du réseau (par exemple, lors du passage aux horaires d'été).
- **Voir les statistiques** : Accès à des tableaux de bord anonymisés sur l'utilisation de l'application, permettant d'analyser les habitudes de déplacement dans la ville.

3.2.2 Diagramme de Séquence



Étapes 1 et 2 (La demande) : L'utilisateur valide sa recherche sur son téléphone.

L'application envoie instantanément la demande au serveur central.

Étape 3 (Le réflexe de la mémoire) : Avant de lancer des calculs mathématiques lourds, le serveur interroge la mémoire rapide (le Cache Redis). Il vérifie si un autre utilisateur n'aurait pas demandé *exactement le même trajet* dans les 30 dernières secondes.

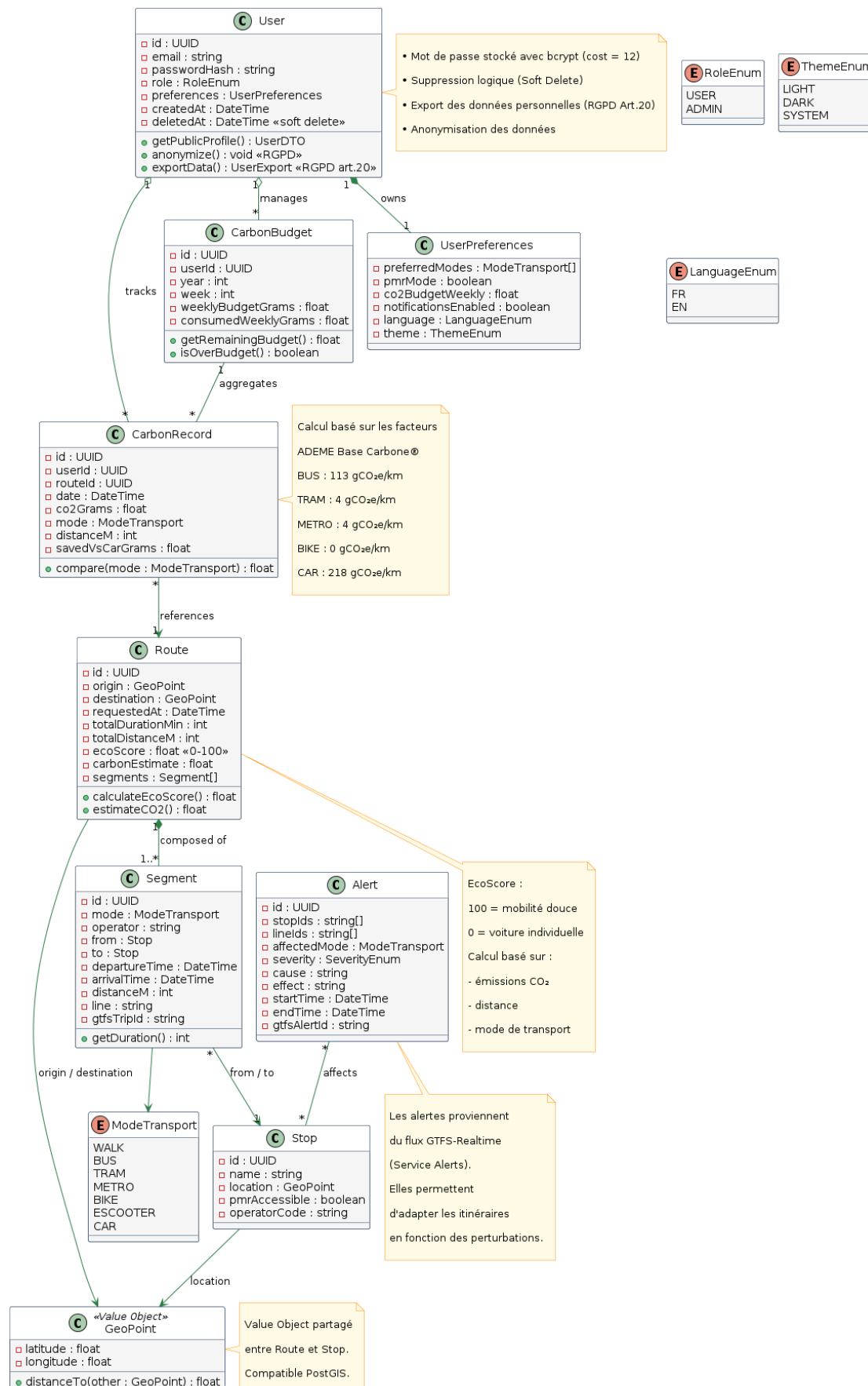
Alternative A : Le trajet est connu (Le chemin rapide) : Si la réponse est oui, le serveur récupère le résultat et l'affiche instantanément à l'utilisateur.

Alternative B : Le trajet est nouveau (Le chemin complet) :

- Le serveur interroge la base cartographique (PostgreSQL) pour trouver physiquement le chemin le plus court dans les rues de la ville.
- Il interroge ensuite les serveurs de la ville (Opérateur Transport) pour savoir si un bus est en retard ou si une route est barrée.
- Il fusionne ces deux informations (la carte théorique + la réalité en direct).
- Il sauvegarde ce résultat tout neuf dans la mémoire rapide pour les 30 prochaines secondes.

Étapes 12 et 13 (L'affichage) : Le serveur renvoie l'itinéraire final à l'application, qui dessine la carte et affiche les options (durée, empreinte carbone) sur l'écran de l'utilisateur.

3.1.3 Diagramme de classes



L'architecture se divise autour de trois grandes familles de classes :

A. Gestion des Utilisateurs (Comptes et Sécurité)

- **User** : Représente le compte de l'utilisateur. Il gère l'identification sécurisée (mot de passe crypté) et les obligations légales (anonymisation, export des données).
- **UserPreferences** : Regroupe les paramètres personnalisés de l'utilisateur, comme le choix du thème (clair/sombre), la langue, ou l'activation du mode PMR pour n'afficher que des trajets accessibles.

B. Le Moteur de Mobilité (Calcul d'itinéraires)

- **Route (Itinéraire)** : Représente un trajet complet d'un point A à un point B. C'est lui qui consolide la durée totale, la distance et qui attribue un "EcoScore" de 0 à 100.
- **Segment** : Un itinéraire est souvent découpé en plusieurs morceaux (ex: marcher jusqu'à l'arrêt, puis prendre le bus ...). Chaque morceau est un "Segment".
- **Stop (Arrêt/Station)** : Représente un lieu physique sur le réseau (arrêt de bus, station de métro), avec ses coordonnées géographiques.
- **GeoPoint** : Un petit objet technique qui stocke simplement la latitude et la longitude pour s'intégrer facilement avec la base cartographique PostGIS.
- **Alert** : Permet de stocker les informations en temps réel sur les pannes, travaux ou retards (données GTFS) pour informer l'utilisateur.

C. Le Module Écologique (Empreinte Carbone)

- **CarbonRecord (Historique)** : Enregistre l'empreinte carbone (en grammes de CO2) de chaque trajet effectué, en la comparant avec ce qu'aurait coûté le même trajet en voiture individuelle.
- **CarbonBudget** : Un outil de suivi hebdomadaire permettant à l'utilisateur de se fixer un plafond d'émissions à ne pas dépasser.

3.3 Sécurité et conformité

Authentification : L'authentification repose sur des jetons JWT. Après validation de ses identifiants, l'utilisateur reçoit un jeton signé lui permettant d'accéder aux fonctionnalités sécurisées de l'application.

Chiffrement des mots de passe : Les mots de passe ne sont jamais stockés en clair. Ils sont hachés à l'aide de l'algorithme **bcrypt** afin de limiter les attaques par force brute.

Gestion des rôles: Le système distingue plusieurs profils (utilisateur, administrateur) afin de contrôler précisément les autorisations d'accès.

Conformité RGPD : UrbanFlow Mobility applique le principe du *Privacy by Design*. Les principales mesures mises en œuvre sont :

- minimisation des données collectées
- export des données personnelles
- suppression logique (*soft delete*)
- hébergement des données au sein de l'Union européenne

Protection OWASP : Les principales vulnérabilités identifiées par l'OWASP sont prises en compte :

- protection contre les injections SQL
- protection XSS
- protection CSRF lorsque nécessaire
- limitation des tentatives de connexion
- journalisation des erreurs

3.4 Performances et éco-conception

Progressive Web App : une installation sur smartphone

Optimisations Frontend :

- chargement différé (*Lazy Loading*) des composants
- optimisation automatique des images avec Next.js

Démarche Green IT :

- utilisation majoritaire de technologies open source
- réduction du volume de données échangées
- mutualisation des traitements
- limitation des appels vers les APIs externes grâce au cache

4 – Bilan du projet

Au terme de cette phase de conception, le projet UrbanFlow Mobility s'appuie désormais sur des fondations techniques et fonctionnelles particulièrement solides. L'ambition initiale était d'offrir aux citoyens une plateforme de mobilité intelligente, fluide et éco-responsable. Cet objectif a été traduit avec succès en une architecture logicielle concrète, moderne et parfaitement réalisable.

Ce travail préparatoire a permis de sécuriser trois enjeux majeurs avant même d'écrire la première ligne de code :

- **L'excellence technique et l'évolutivité** : Le choix d'une architecture décentralisée (séparation entre l'interface Vercel ultra-rapide et le backend robuste sous OVHcloud) garantit que la plateforme pourra absorber les pics de trafic sans ralentissement.
- **L'innovation environnementale au cœur du système** : L'isolation du calcul de l'empreinte carbone via un microservice dédié (FastAPI) prouve que l'écologie n'est pas une simple option d'affichage, mais un moteur fondamental de l'application, capable d'évoluer de manière autonome.
- **La maîtrise totale des risques et de la conformité** : La sélection d'outils stricts (PostgreSQL pour l'intégrité spatiale, typage fort) couplée à une intégration native des exigences du RGPD assure une plateforme fiable, légale et sécurisée pour la métropole et ses habitants.

Loin d'être une simple étude théorique, l'ensemble des livrables produits (modélisation UML, diagrammes de flux, benchmarks technologiques) constitue aujourd'hui une véritable feuille de route opérationnelle.

Toutes les zones d'ombre ayant été levées et les choix technologiques validés, UrbanFlow Mobility est désormais paré pour entrer sereinement dans sa phase de développement (MVP). Les équipes techniques disposent d'un cadre clair et sans ambiguïté pour développer, tester et déployer progressivement cette solution au service des usagers.

4.1 Difficultés rencontrées

La phase de conception a mis en lumière certains défis majeurs, à commencer par la modélisation du calculateur d'empreinte carbone. Il a fallu analyser minutieusement les données complexes de la Base Carbone de l'ADEME afin de les transformer en estimations environnementales qui soient à la fois scientifiquement rigoureuses et parfaitement claires pour l'utilisateur.

Par ailleurs, l'architecture retenue représente un véritable défi d'intégration. Le projet fait dialoguer de nombreuses technologies de pointe (Next.js, NestJS, FastAPI, Redis PostgreSQL/PostGIS, et Docker). La communication entre ces différents écosystèmes exigera une orchestration minutieuse et des tests stricts tout au long du développement.

Loin d'être des freins, ces exigences ont permis d'identifier, d'anticiper et de désamorcer les principaux risques technologiques bien avant l'écriture des premières lignes de code, garantissant ainsi un passage à la réalisation en toute sérénité.

4.2 Limites du projet

Une forte dépendance aux données externes (Opérateurs et ADEME) : La précision des itinéraires en temps réel et des alertes repose intégralement sur la qualité des flux fournis par les opérateurs de transport (API GTFS-RT). En cas de panne de leurs serveurs ou de données erronées à la source, l'application répercutera inévitablement ces erreurs. De même, le calcul de l'empreinte carbone dépend de la disponibilité de la Base Carbone de l'ADEME.

La personnalisation avancée des itinéraires ne pourra être pleinement évaluée qu'après la phase de développement.

4.3 Perspectives d'évolution

Le projet actuel répond aux besoins essentiels exprimés par le commanditaire, plusieurs évolutions pourront être envisagées à long terme afin d'enrichir progressivement la plateforme. Parmi les principales perspectives d'évolution figurent :

- **L'intégration du paiement des titres de transport**, permettant aux usagers d'acheter directement leurs billets ou abonnements depuis l'application, sans passer par une plateforme externe.
- **La réservation de vélos, trottinettes ou véhicules en libre-service**, afin de transformer UrbanFlow Mobility en une véritable plateforme *Mobility as a Service (MaaS)* centralisant l'ensemble des services de mobilité.
- **La mise en place d'un système de fidélité**, récompensant les utilisateurs privilégiant les modes de transport les plus respectueux de l'environnement. Des points ou badges pourraient être attribués en fonction des émissions de CO₂ économisées, encourageant ainsi l'adoption de comportements plus durables.

En complément, plusieurs évolutions techniques pourront également être étudiées :

- l'intégration d'algorithmes d'intelligence artificielle afin d'anticiper les perturbations du réseau et de proposer des itinéraires prédictifs
- l'ajout de nouveaux opérateurs de mobilité (autopartage, parkings relais, bornes de recharge pour véhicules électriques)
- l'adaptation de la plateforme à d'autres collectivités territoriales, permettant son déploiement dans plusieurs métropoles françaises.

Ces évolutions s'inscrivent dans une démarche d'amélioration continue et démontrent que l'architecture retenue est suffisamment flexible pour accompagner les futurs besoins de la mobilité urbaine.

5 - Conclusion

Le projet UrbanFlow Mobility a atteint son objectif premier : concevoir les fondations solides d'une plateforme de mobilité multimodale, écologique et axée sur le temps réel, adaptée aux besoins d'une métropole de 500 000 habitants.

La démarche adoptée a permis de valider chaque étape du cycle de vie logiciel. De l'analyse fonctionnelle stricte à la modélisation UML, jusqu'au choix d'une architecture moderne et découpée (Next.js, NestJS, FastAPI, PostgreSQL/PostGIS), cette phase de conception garantit le développement d'un produit (MVP) performant, sécurisé et capable d'absorber une montée en charge. Ce projet prouve également qu'il est possible d'allier innovation technique et respect des normes exigeantes du web d'aujourd'hui (RGPD, accessibilité, éco-conception).

Bien que ce MVP répond fidèlement au cahier des charges actuel, son architecture ouverte prépare déjà l'avenir de la plateforme. De nombreuses évolutions sont envisageables pour transformer cette application en un véritable guichet unique : l'intégration du paiement des titres de transport, l'utilisation de l'intelligence artificielle pour anticiper l'affluence, ou encore l'ajout de fonctionnalités communautaires (signalement d'incidents par les usagers).

En définitive, UrbanFlow Mobility s'est révélé être un projet de synthèse décisif. Il illustre concrètement ma capacité à piloter un produit numérique de bout en bout, en conciliant les attentes des usagers, les contraintes technologiques et les enjeux écologiques de demain. Cette réalisation valide pleinement mes compétences de **Concepteur Développeur de Solutions Digitales** et constitue un tremplin solide vers des projets professionnels de plus grande envergure.

Webographie

Développement

- Next.js Documentation — <https://nextjs.org/docs>
- NestJS Documentation — <https://docs.nestjs.com>
- FastAPI Documentation — <https://fastapi.tiangolo.com>
- PostgreSQL Documentation — <https://www.postgresql.org/docs>
- PostGIS Documentation — <https://postgis.net/documentation>
- Redis Documentation — <https://redis.io/docs>
- Docker Documentation — <https://docs.docker.com>

Open Data

- GTFS Reference — <https://gtfs.org>
- GTFS-Realtime — <https://gtfs.org/documentation/realtime>
- OpenStreetMap — <https://www.openstreetmap.org>
- Base Carbone® ADEME — <https://basecarbone.ademe.fr>

Accessibilité et sécurité

- RGAA — <https://accessibilite.numerique.gouv.fr>
- WCAG 2.1 — <https://www.w3.org/WAI/standards-guidelines/wcag>
- OWASP Top 10 — <https://owasp.org/www-project-top-ten>
- CNIL — <https://www.cnil.fr>

Liens

Solution :

Github :
